

SQLP 과목 III 튜닝 스타일 모의문제 · 2회차

SQL 자격검정 실전문제 3장 스타일 · 4지선다 · 총 20문항 · 원본 창작

1

DB 버퍼 캐시, Redo 로그 버퍼, Undo(롤백) 세그먼트가 데이터 변경 과정에서 담당하는 역할에 대한 설명으로 가장 적절하지 않은 것은?

- ① Undo 세그먼트에 보관된 이전 이미지는 트랜잭션 롤백에 쓰일 뿐 아니라, 인스턴스 장애 복구 시 커밋된 변경을 디스크에 다시 반영하는 roll forward의 입력으로도 사용된다.
- ② Redo 로그 버퍼에는 변경을 그대로 재현할 수 있는 변경 벡터(change vector)가 쌓이며, 이 내용은 로그 파일로 기록되어 인스턴스 장애 시 커밋된 변경을 다시 적용하는 roll forward의 근거가 된다.
- ③ Undo 세그먼트에는 블록을 변경하기 전의 이전 이미지(before image)가 저장되어, 트랜잭션 롤백과 다른 세션의 읽기 일관성(변경 전 데이터 재구성)에 쓰인다.
- ④ DB 버퍼 캐시는 디스크에서 읽은 데이터 블록을 메모리에 적재해 두는 공간으로, 이후 같은 블록을 다시 참조할 때 물리적 I/O 없이 캐시에서 재사용하게 해 준다.

2

오라클의 블록 I/O 방식(Single Block I/O·Multiblock I/O)과 그때 관측되는 대기 이벤트의 짝으로 가장 적절한 것은?

- ① db file scattered read는 이름 그대로 흩어진 블록을 한 번에 하나씩 임의로 읽는 방식에서 발생하며, 주로 인덱스 Range Scan의 테이블 액세스 단계에서 관측된다.
- ② Full Table Scan처럼 인접한 여러 블록을 한 번의 I/O Call로 묶어 읽는 Multiblock I/O 도중에는 db file sequential read 대기 이벤트가 나타난다.
- ③ 오라클에서 대량 블록을 훑는 Multiblock I/O에 대응하는 대기 이벤트는 SQL Server가 쓰는 PAGEIOLATCH_SH이며, db file scattered read는 SQL Server의 Full Scan 대기 이벤트다.
- ④ 인덱스를 경유해 얻은 ROWID로 테이블 블록을 하나씩 임의로 찾아 읽는 Single Block I/O가 진행되는 동안에는 db file sequential read 대기 이벤트가 관측된다.

한 야간 배치가 스마트 계량기 약 4만 8천 대의 검침값을 검침값 테이블에서 대당 한 건씩 조회·적재한다. 개발자가 계량기번호를 매번 문자열 리터럴로 직접 이어붙여(WHERE 계량기번호 = '001234...') SQL을 생성했고, 배치가 느려 아래 TKPROF를 수집했다. 이 트레이스가 가리키는 주된 문제와 그 처방을 옳게 짚지은 것은?

아 래							
OVERALL TOTALS FOR ALL NON-RECURSIVE STATEMENTS							
Call	Count	CPU Time	Elapsed	Disk	Query	Current	Rows
Parse	48000	42.510	55.320	0	0	0	0
Execute	48000	8.230	9.140	0	96000	0	48000
Fetch	48000	3.120	12.470	820	192000	0	48000
Total	144000	53.860	76.930	820	288000	0	96000
Misses in library cache during parse: 48000							
OVERALL TOTALS FOR ALL RECURSIVE STATEMENTS							
Call	Count	CPU Time	Elapsed	Disk	Query	Current	Rows
Parse	336000	18.640	22.180	0	0	0	0
Execute	384000	15.720	17.960	140	1152000	2400	384000
Fetch	720000	9.480	11.230	260	2160000	0	768000
Total	1440000	43.840	51.370	400	3312000	2400	1152000
Misses in library cache during parse: 336000							

- ① 옵티마이저 통계가 오래돼 실행계획 품질이 나빠진 것이 근본 원인이므로, 통계를 최신화하고 히스토그램을 수집하면 하드파싱이 줄어 파싱 부하가 해소된다.
- ② Disk가 820으로 매우 낮아 물리 I/O는 문제가 없고, 남은 병목은 Fetch의 Elapsed 12.47초이므로 네트워크 왕복 대기를 줄이는 것이 최우선 처방이다.
- ③ 재귀(Recursive) Parse가 33만 6천 회로 비수행문 Parse의 7배다. 재귀 SQL 자체가 근본 원인이므로, 배치가 호출하는 사용자 정의 함수와 트리거를 제거하는 것이 처방이다.
- ④ 비수행문 Parse 4만 8천 회에 라이브러리 캐시 미스도 4만 8천 회 — 파싱이 전부 하드파싱이고 파싱이 비수행문 CPU의 약 79%를 차지한다. 리터럴을 바인드 변수로 바꿔 커서를 공유(소프트파싱)하는 것이 처방이다.

4

결제 테이블(약 5,000만 건)과 가맹점 테이블(약 12만 건)을 조인해 가맹점별 결제금액을 집계하는 SQL의 TKPROF 결과이다. 아래 트레이스에 대한 해석으로 가장 적절한 것은?

아 래							
Call	Count	CPU Time	Elapsed Time	Disk	Query	Current	Rows
Parse	1	0.004	0.004	0	0	0	0
Execute	1	0.000	0.000	0	0	0	0
Fetch	251	6.820	44.300	150000	1250000	0	3000
Total	253	6.824	44.304	150000	1250000	0	3000
Rows	Row Source Operation						
3000	HASH JOIN (cr=1250000 pr=150000 pw=0 time=44210000 us)						
120000	TABLE ACCESS FULL 가맹점 (cr=70000 pr=8000 pw=0 time=1540000 us)						
50000000	TABLE ACCESS FULL 결제 (cr=1180000 pr=142000 pw=0 time=40180000 us)						

- ① CPU Time 6.82초가 Elapsed 44.3초의 대부분을 차지하므로 이 SQL은 CPU 바운드이며, 병렬도(DOP)를 높이는 것이 최우선 처방이다.
- ② 논리적 I/O 125만 블록 중 15만(약 12%)을 디스크에서 읽어 BCHR은 약 88%이며, 블록 소비·물리 읽기·소요 시간이 모두 결제 Full Scan(cr=1,180,000, pr=142,000, time≈40.2초)에 집중되므로 병목은 결제 테이블 액세스다.
- ③ BCHR이 88%로 낮으니 버퍼 캐시 부족이 근본 원인이고, 버퍼 캐시를 키우면 결제 Full Scan의 블록 I/O가 사라진다.
- ④ Row Source에 인덱스 스캔이 없으니 가맹점 테이블(12만 건)의 인덱스 부재가 병목이며, 가맹점에 인덱스를 만들면 44초가 해소된다.

5

예상(추정) 실행계획을 확인하는 여러 도구의 문장 수행 여부에 대한 설명으로 가장 적절한 것은?

- ① 오라클의 SET AUTOTRACE TRACEONLY(EXPLAIN·STATISTICS를 함께 켜 상태)는 결과 행을 화면에 출력하지 않으므로, 대상 SELECT문 자체도 전혀 수행하지 않는다.
- ② SQL Server의 SET SHOWPLAN_TEXT ON은 대상 문장을 실제로 수행한 뒤 실측 통계가 담긴 실제 실행계획을 반환한다.
- ③ SQL Server의 SET SHOWPLAN_ALL ON은 이후 문장을 수행하지 않고 예상 실행계획만 반환하는 반면, SET STATISTICS PROFILE ON은 문장을 실제로 수행한 뒤 실측 행 수가 포함된 실제 실행계획을 반환한다.
- ④ 오라클의 EXPLAIN PLAN FOR ...는 문장을 수행하면서 실측 행 수까지 PLAN_TABLE에 적재하므로, DBMS_XPLAN.DISPLAY로 조회하면 실측치가 반영된 실제 실행계획을 볼 수 있다.

6

통신 요금 청구내역 테이블(약 740만 건)에는 결합 인덱스 청구_IX = (요금제코드, 청구월)가 있다. 요금제명을 함께 얻기 위해 요금제 테이블(기본키 요금제_PK = 요금제코드)과 조인하는 다음 SQL의 실행계획이다. 이 실행계획에 대한 설명으로 가장 적절한 것은?

아래

Id	Pid	Operation	(Cost	Card	Bytes)
0	-	SELECT STATEMENT	(312	2100	113400)
1	0	NESTED LOOPS	(312	2100	113400)
2	1	TABLE ACCESS BY INDEX ROWID 청구내역	(307	2100	75600)
3	2	INDEX RANGE SCAN 청구_IX	(22	8400	)
4	1	TABLE ACCESS BY INDEX ROWID 요금제	(1	1	18)
5	4	INDEX UNIQUE SCAN 요금제_PK	(0	1	)

- ① 요금제코드·청구월·납부상태 세 조건이 모두 청구_IX의 인덱스 액세스 조건에 포함되어 INDEX RANGE SCAN 단계에서 함께 처리된다.
- ② 요금제코드(등치)와 청구월(범위)은 청구_IX를 통한 인덱스 액세스 조건으로 처리되고, 인덱스에 없는 납부상태는 TABLE ACCESS BY INDEX ROWID 단계에서 필터로 걸러진다.
- ③ INDEX RANGE SCAN이 반환한 8,400건이 곧 이 SQL의 최종 결과 건수이며, TABLE ACCESS BY INDEX ROWID 단계에서는 건수 감소가 일어나지 않는다.
- ④ 선두 컬럼 요금제코드에 조건을 주지 않고 청구월 범위만 주더라도 청구_IX의 INDEX RANGE SCAN은 그대로 성립한다.

7

예약 테이블(약 900만 건, 세그먼트 약 90,000 블록)에서 예약_IX 인덱스를 경유해 특정 기간의 예약을 조회한 SQL의 트레이스이다. 이 트레이스에 대한 옳은 진단은?

아래

Call	Count	CPU Time	Elapsed Time	Disk	Query	Current	Rows
Parse	1	0.006	0.007	0	0	0	0
Execute	1	0.000	0.000	0	0	0	0
Fetch	6201	5.640	33.900	50000	211340	0	620000
Total	6203	5.646	33.907	50000	211340	0	620000
Rows	Row Source Operation						
620000	TABLE ACCESS BY INDEX ROWID 예약 (cr=211340 pr=50000 pw=0 time=33610000 us)						
620000	INDEX RANGE SCAN 예약_IX (cr=1340 pr=340 pw=0 time=642000 us)						

- ① 인덱스 스캔(cr=1,340)이 효율적이므로 인덱스는 문제가 없고, 병목은 Fetch를 6,201회로 나눈 Array Size이니 Array Size만 키우면 33초가 해소된다.
- ② 테이블 랜덤 액세스가 ROWID 62만 건에 블록 I/O 21만을 써 CF가 테이블 블록 수(90,000)의 약 34배로 나쁘고, 읽은 행이 전체의 약 7%인데도 단일블록 랜덤 읽기 21만 > Full Scan 9만 블록이라 손익분기점을 넘었다. Full Scan(또는 인덱스 커버링)이 유리하다.
- ③ ROWID당 블록 읽기가 약 0.34로 1보다 작으니 클러스터링 팩터는 최상급이고, 33초 대기는 인덱스 리프 블록 분할(cr=1,340) 때문이다.
- ④ 물리 읽기 50,000이 논리 읽기 211,340의 약 24%로 BCHR이 약 76%이므로 버퍼 캐시 부족이 근본 원인이고, 캐시를 키우면 이 SQL이 해결된다.

8

인덱스가 정상적인 Range Scan으로 사용되기 위한 조건에 대한 설명으로 가장 적절하지 않은 것은?

- ① 인덱스 선두 컬럼을 SUBSTR·NVL 같은 함수로 감싸거나 산술 연산을 적용하면, 가공된 값의 정렬 순서를 인덱스가 알 수 없어 스캔 시작·끝점을 잡지 못한다.
- ② 문자형 컬럼을 숫자 상수와 비교하면(예: char_col = 100) 오라클은 우선순위가 높은 숫자에 맞춰 컬럼 쪽을 TO_NUMBER로 묵시적 형변환하며, 이는 컬럼을 가공한 것과 같아 그 인덱스를 정상적으로 타지 못한다.
- ③ 단일 컬럼 인덱스에서 IS NULL 조건은 NULL이 인덱스에 저장되지 않아 그 인덱스로 대상 행을 찾기 어렵고, !=·NOT IN 같은 부정형도 스캔 시작점을 확정하지 못해 정상 Range Scan에 부적합하다.
- ④ 인덱스 컬럼에 함수를 씌우거나 양변에 같은 연산을 걸어도, 옵티마이저가 조건을 인덱스가 활용 가능한 형태로 되돌려 주므로 Range Scan은 그대로 유지된다.

9

결합 인덱스 (A, B)에서 선행 컬럼의 조건 종류에 따라 후행 컬럼이 스캔 범위에 어떻게 기여하는지에 대한 설명으로 가장 적절하지 않은 것은?

- ① 선행 컬럼 A가 범위 조건이더라도 후행 컬럼 B에 등치 조건이 있으면, B가 스캔 시작·끝점을 확정해 A 범위 전체가 아니라 좁은 구간만 읽게 되므로 액세스 범위가 최소화된다.
- ② 선행 컬럼 A가 범위 조건이면 A가 여러 값에 걸쳐 열리므로, 후행 컬럼 B의 조건은 스캔 시작·끝점을 정하는 액세스 조건이 되지 못하고 리프 엔트리를 걸러 내는 필터 조건으로만 작동한다.
- ③ Index Skip Scan은 선행 컬럼의 서로 다른 값 개수가 적고 후행 컬럼에 조건이 주어질 때, 선행 컬럼의 각 값 구간을 건너뛰며 후행 조건을 탐색해 선행 컬럼이 조건에 없어도 인덱스를 활용하는 예외적 경로다.
- ④ A에 등치(=), B에 범위(> 등) 조건이 주어지면, A가 한 점으로 고정된 뒤 그 안에서 B의 범위가 정렬 순서대로 이어져 인덱스 스캔 범위가 효율적으로 좁혀진다.

10

인사평가 결과를 등급으로 환산하기 위해, 평가점수 테이블(약 640만 건)의 평가점수가 등급기준 테이블(등급 구간 8건)의 하한점수와 상한점수 사이에 드는지로 조인하는 SQL의 실행계획이다. 두 테이블 어디에도 조인에 쓸 인덱스는 없다. 이 실행계획에 대한 설명으로 가장 적절한 것은?

아 래

```
SELECT e.사번, e.평가점수, g.등급코드, g.등급명
FROM 평가점수 e, 등급기준 g
WHERE e.평가점수 >= g.하한점수
AND e.평가점수 <= g.상한점수;
```

- ① 양쪽 집합을 각각 정렬(SORT JOIN)한 뒤 정렬 순서를 따라 병합(MERGE JOIN)하는 소트 머지 조인이며, 조인 조건이 범위(비등치)여서 해시 조인이 쓰일 수 없는 경우에 적합하다.
- ② SORT JOIN 노드가 두 개 있으므로 이는 정렬된 결과를 해시 버킷에 적재해 탐침하는 해시 조인의 한 형태이다.
- ③ 두 SORT JOIN 노드가 정렬 부하를 만드므로, 이 방식은 인덱스를 경유하는 NL 조인보다 항상 느리다.
- ④ MERGE JOIN은 등치(=) 조인만 처리할 수 있으므로, 이 플랜의 조인 조건도 실제로는 등치 조건으로 변환되어 수행된 것이다.

11

해시 조인(Hash Join)의 수행 원리와 Build/Probe 단계에 대한 설명으로 가장 적절하지 않은 것은?

- ① Build 단계에서 한쪽 입력을 읽어 조인 키로 Hash Area에 해시 테이블을 만들고, Probe 단계에서 다른 쪽 입력을 한 행씩 읽어 같은 해시 함수로 버킷을 찾아 매칭한다.
- ② 옵티마이저는 두 입력 중 크기가 더 클 것으로 추정되는 쪽을 Build Input으로 삼아, Probe Input을 반복 탐색하는 횟수를 줄이려 한다.
- ③ 해시 조인은 조인 키를 해시 함수로 같은 버킷에 모으는 방식이므로, 등치(=) 조인 조건에서 성립한다.
- ④ Build Input이 Hash Area 크기를 넘어서면 일부 파티션을 Temp 테이블스페이스에 기록하는 one-pass·multi-pass 방식으로 처리되어 성능이 저하될 수 있다.

12

옵티마이저의 쿼리 변환(Query Transformation) 중 서브쿼리 Unnesting과 조건절 Pushing에 대한 설명으로 가장 적절한 것은?

- ① 서브쿼리 Unnesting과 View Merging은 둘 다 두 쿼리 블록을 하나로 합치는 작업이므로, 사실상 같은 변환을 부르는 다른 이름이다.
- ② 조건절 Pushing은 뷰 바깥의 조건을 이용해 조인 순서를 한 방향으로 고정함으로써, 옵티마이저가 다른 조인 순서를 탐색하지 못하도록 막는 변환이다.
- ③ 조건절 이행(Transitive Predicate Generation)은 서브쿼리 Unnesting의 다른 이름으로, 중첩 서브쿼리를 조인으로 바꾸는 과정을 가리킨다.
- ④ 조건절 Pushing은 인라인 뷰 바깥에 있던 조건을 뷰 내부로 밀어 넣어, 뷰가 처리해야 할 행을 미리 줄이는 변환이다.

13

세 세션이 논리적으로 같은 상품 목록을 얻으려고 아래 SQL을 각각 수행했다. 세 SQL은 키워드 대소문자·힌트·공백만 다르고 조회 결과는 동일하다. 수행 직후 라이브러리 캐시(v\$sql)를 요약한 결과가 함께 주어졌다. 커서 공유(Cursor Sharing)에 대한 설명으로 가장 적절하지 않은 것은?

아 래

```
-- 세션 P
select prod_nm, unit_price from prod where category_cd = 'A7';

-- 세션 Q (키워드만 대문자)
SELECT prod_nm, unit_price FROM prod WHERE category_cd = 'A7';

-- 세션 R (인덱스 힌트 추가)
select /*+ index(prod prod_x01) */ prod_nm, unit_price
from prod where category_cd = 'A7';
```

- ① 오라클은 라이브러리 캐시에서 커서를 찾기 전에 SQL 텍스트를 대문자로 정규화하고 힌트·주석·여백을 제거하므로, 세 SQL은 하나의 부모 커서를 공유한다.
- ② 힌트도 SQL 텍스트의 일부이므로, /*+ index(...) */를 붙인 세션 R은 원본(세션 P)과 별개의 커서(s2v9...)를 사용한다.
- ③ 세 커서를 하나로 합쳐 소프트파싱하려면 리터럴 'A7'을 바인드 변수로 바꾸는 것만으로는 부족하고, 대소문자·힌트·공백까지 동일하게 맞춰 텍스트를 일치시켜야 한다.
- ④ 세 SQL은 키워드 대소문자·힌트·공백만 다를 뿐 의미는 같지만, 텍스트가 문자 단위로 일치하지 않아 서로 다른 SQL_ID로 각각 하드파싱(LOADS 1)되었다.

14

옵티마이저 힌트(Hint)의 성격과 동작에 대한 설명으로 가장 적절한 것은?

- ① LEADING과 ORDERED는 각 테이블을 어떤 인덱스로 읽을지, 즉 액세스 경로를 지정하는 힌트다.
- ② USE_NL(A, B) 힌트를 지정하면 실행 가능 여부와 무관하게 항상 A와 B가 NL 조인으로 수행된다.
- ③ 오타·문법 오류로 유효하지 않은 힌트나, 문법상 유효하더라도 그 상황에서 실행 불가능한 경로를 지시하는 힌트는 오류를 일으키지 않고 무시된다.
- ④ 힌트에 오타가 있으면 옵티마이저가 구문 오류를 발생시켜 SQL이 실패하므로, 힌트는 문법을 정확히 지켜야 실행된다.

15

소트(Sort) 튜닝과 소트 오퍼레이션에 대한 설명으로 가장 적절하지 않은 것은?

- ① 인덱스는 키 순서로 정렬돼 있지만, 그 순서로 ORDER BY를 처리할 때 옵티마이저는 결과의 정합성을 보장하려고 SORT ORDER BY 오퍼레이션을 한 번 더 추가한다.
- ② 정렬은 PGA의 Sort Area에서 수행되며, 정렬 대상이 Sort Area에 다 담기지 못하면 Temp 테이블스페이스를 쓰는 디스크 소트(multi-pass)로 처리되어 성능이 나빠진다.
- ③ GROUP BY 컬럼이 인덱스 선두 컬럼의 정렬 순서와 맞으면, 옵티마이저는 SORT GROUP BY 없이 인덱스를 훑는 것만으로 그룹핑을 처리할 수 있다.
- ④ 실행계획의 SORT UNIQUE는 중복 제거를 위한, SORT AGGREGATE는 GROUP BY 없는 전체 집계를 위한 소트 계열 오퍼레이션이다.

16

배치 프로그램이 이벤트 로그 150만 건을 아래 SQL로 전량 Fetch해 파일로 출력한다. 스칼라 서브쿼리로 장비명을 함께 조회하며, dev_id의 서로 다른 값(distinct)은 40종뿐이다. 수행 통계가 함께 주어졌을 때 데이터베이스 Call 최소화에 대한 해석·처방으로 가장 적절한 것은?

아 래

```
SELECT e.evt_id,
       e.evt_dtm,
       e.dev_id,
       (SELECT d.dev_nm FROM device d WHERE d.dev_id = e.dev_id) AS dev_nm
FROM   event_log e
WHERE  e.evt_dtm >= DATE '2026-06-01';
```

- ① Fetch Call이 3,001회이므로 ArraySize는 약 50이며, ArraySize를 키우면 device 조회 횟수가 줄어 스칼라 서브쿼리 캐싱 효과가 커진다.
- ② 스칼라 서브쿼리 캐싱이 Fetch Call 횟수를 3,001회에서 줄여 네트워크 왕복을 감소시키고, ArraySize를 조정하면 device 조회가 40회 수준으로 줄어든다.
- ③ ArraySize는 $1,500,000 \div (3,001 - 1) = 500$ 이며, ArraySize를 더 키우면 Fetch Call(User Call) 횟수가 줄어 왕복이 감소한다. 한편 스칼라 서브쿼리 캐싱은 dev_id가 40종뿐이라 device 조회를 최대 40회 수준으로 줄인다.
- ④ ArraySize와 무관하게 Fetch Call은 반환 행 수와 항상 같으므로, 왕복을 줄이려면 스칼라 서브쿼리 캐싱만이 유효하다.

17

센서측정 테이블은 측정일시(DATE 타입)를 기준으로 분기별 RANGE 파티셔닝되어 있다(아래 DDL). 각 섹터지는 SELECT AVG(측정값) FROM sensor_log WHERE ... 형태의 조회에 대한 설명이다. Partition Pruning이 효과적으로 작동해 필요한 파티션만 읽는 것으로 보기 가장 적절하지 않은 것은?

아래

```
CREATE TABLE sensor_log (
  측정id NUMBER,
  측정일시 DATE,
  센서번호 NUMBER,
  측정값 NUMBER
)
PARTITION BY RANGE (측정일시) (
  PARTITION sp_2025q1 VALUES LESS THAN (DATE '2025-04-01'),
  PARTITION sp_2025q2 VALUES LESS THAN (DATE '2025-07-01'),
  PARTITION sp_2025q3 VALUES LESS THAN (DATE '2025-10-01'),
  PARTITION sp_2025q4 VALUES LESS THAN (DATE '2026-01-01'),
  PARTITION sp_2026q1 VALUES LESS THAN (DATE '2026-04-01'),
  PARTITION sp_max VALUES LESS THAN (MAXVALUE)
);
```

- ① WHERE 측정일시 >= DATE '2025-07-01' AND 측정일시 < DATE '2025-10-01' 은 sp_2025q3 한 파티션만 읽고 나머지는 건너뛴다.
- ② WHERE 측정일시 BETWEEN DATE '2025-01-01' AND DATE '2025-06-30' 은 sp_2025q1·sp_2025q2 두 파티션만 접근한다.
- ③ WHERE 측정일시 >= DATE '2026-01-01' 은 sp_2026q1·sp_max만 스캔하고 2025년 파티션은 모두 건너뛴다.
- ④ WHERE TRUNC(측정일시) = DATE '2025-08-15' 는 8월 15일 하루만 겨냥하므로 옵티마이저가 sp_2025q3 한 파티션만 읽어 Pruning이 작동한다.

18

광고 노출이력 테이블(약 3억 건, 8개 파티션)과 캠페인 테이블(약 5천 건)을 병렬로 해시 조인하는 야간 정산 배치 SQL의 실행계획이다. 이 병렬 실행계획은 두 테이블을 병렬 서버(Slave) 사이에 어떻게 분배하고 있는가? 가장 정확히 설명한 것은?

아래

	Id	Operation	Name	Pstart	Pstop	TQ	IN-OUT
	0	SELECT STATEMENT					
	1	PX COORDINATOR					
	2	PX SEND QC (RANDOM)	:TQ10001			Q1,01	P->S
*	3	HASH JOIN				Q1,01	PCWP
	4	PX RECEIVE				Q1,01	PCWP
	5	PX SEND BROADCAST	:TQ10000			Q1,00	P->P
	6	PX BLOCK ITERATOR				Q1,00	PCWC
	7	TABLE ACCESS FULL	캠페인			Q1,00	PCWP
	8	PX BLOCK ITERATOR		1	8	Q1,01	PCWC
*	9	TABLE ACCESS FULL	노출이력	1	8	Q1,01	PCWP

- ① 노출이력과 캠페인을 양쪽 다 조인 키 해시로 재분배(hash-hash)하여, 각 서버가 자기 해시 버킷 범위의 양쪽 행만 조인한다.
- ② 캠페인(소형)을 병렬 서버 전체에 복제(broadcast)하고, 노출이력(대형)은 재분배 없이 각 서버가 PX BLOCK ITERATOR로 자기 파티션 조각만 스캔한다(broadcast-none).
- ③ 노출이력을 조인 키 해시로 재분배하고, 캠페인을 병렬 서버 전체에 복제한다(hash-broadcast).
- ④ 노출이력(대형)을 병렬 서버 전체에 복제(broadcast)하고, 캠페인(소형)은 각 서버가 자기 조각만 스캔한다.

19

SQL Server에서 쿠폰 잔여수량 테이블의 이벤트 쿠폰 'CPN-9' 행(초기 잔여수량 = 200)을 두 세션이 아래 순서로 "읽고-계산해-되쓰는(read-modify-write)" 방식으로 각각 1씩 차감한다. 두 세션이 t4까지 모두 커밋한 뒤의 최종 잔여수량과 그 현상에 대한 진단으로 가장 적절한 것은? (단, 기본 격리수준 READ COMMITTED를 유지하며, 애플리케이션이 UPDATE 시 재조회 없이 앞서 SELECT로 읽은 값을 그대로 사용한다.)

아래		
시각	TX1	TX2
t1	SELECT 잔여수량 FROM 쿠폰 WHERE 쿠폰코드='CPN-9'; → 200 읽음	
t2		SELECT 잔여수량 FROM 쿠폰 WHERE 쿠폰코드='CPN-9'; → 200 읽음
t3	UPDATE 쿠폰 SET 잔여수량 = 200 - 1 WHERE 쿠폰코드='CPN-9'; COMMIT; (→199)	
t4		UPDATE 쿠폰 SET 잔여수량 = 200 - 1 WHERE 쿠폰코드='CPN-9'; COMMIT; (→199)
초기 잔여수량 = 200		

- ① 두 세션의 SELECT가 서로 다른 값을 읽어 발생한 Phantom Read이며, 최종 잔여수량은 198이다.
- ② READ COMMITTED에서는 SELECT가 공유(S) 락을 트랜잭션 끝까지 유지하므로 Lost Update가 원천 차단되어 최종 잔여수량은 198이다.
- ③ 두 세션이 모두 갱신 전 값 200을 읽고 각자 199로 되 썼기 때문에 TX1의 차감이 사라진 Lost Update이며, 최종 잔여수량은 199이다.
- ④ 두 SELECT가 모두 '커밋된 값 200'만 읽었다는 점이 판별 근거이며, RCSI(READ_COMMITTED_SNAPSHOT)를 켜면 최종 잔여수량이 198로 바뀐다.

20

오라클에서 포인트원장 테이블의 같은 회원 적립 행을 두 세션(SID 142, 167)이 동시에 UPDATE하던 중 v\$sqllock을 조회한 결과이다. LMODE·REQUEST는 락 모드 번호(3=RX/Row Exclusive, 6=X/Exclusive, 0=None), BLOCK=1은 이 락이 다른 세션을 막고 있음을 뜻한다. 이 락 상태에 대한 해석으로 가장 적절하지 않은 것은?

아래						
SID	TYPE	ID1	ID2	LMODE	REQUEST	BLOCK
142	TM	88231	0	3 (RX)	0 (None)	0
167	TM	88231	0	3 (RX)	0 (None)	0
142	TX	655370	942	6 (X)	0 (None)	1
167	TX	655370	942	0 (None)	6 (X)	0

- ① TM 락은 두 세션이 모두 RX(모드 3)로 함께 보유하고 있고 RX끼리는 호환되므로, 테이블 수준에서는 서로 대기가 없다.
- ② SID 167은 TX 자원(655370,942)에 X(모드 6)를 요청했으나 LMODE가 None이고 REQUEST가 6이어서 대기 중이며, BLOCK=1인 SID 142에 막혀 있다.
- ③ TX 락은 공유(S) 모드라 서로 호환되므로, SID 142가 SID 167의 커밋을 기다리는 방향의 블로킹이다.
- ④ 같은 행을 두 세션이 갱신하려다 생긴 행 수준 TX 락 경쟁이며, 대기는 SID 167 한쪽에만 있으므로 교착(deadlock)이 아니다.

정답 및 해설

■ 정답 모아보기

1. ①	2. ④	3. ④	4. ②	5. ③
6. ②	7. ②	8. ④	9. ①	10. ①
11. ②	12. ④	13. ①	14. ③	15. ①
16. ③	17. ④	18. ②	19. ③	20. ③

문제 1. 정답 ①

Redo와 Undo는 이름이 비슷해 헷갈리지만, 서로 독립된 축을 담당하는 별개의 구조입니다. 방향으로 갈라 두면 명확합니다.

Redo(변경 벡터) → 앞으로 다시 적용 → roll forward (복구 시 커밋분 재현)

Undo(이전 이미지) → 뒤로 되돌리기 → rollback + 읽기 일관성(변경 전 재구성)

Redo는 "무엇을 어떻게 바꿨는가"를 재현 가능한 형태로 담습니다. 인스턴스가 비정상 종료된 뒤 재기동하면, 아직 데이터파일에 반영되지 못한 커밋된 변경을 Redo로 다시 적용(roll forward) 합니다.

Undo는 "바꾸기 전에는 무엇이었는데"를 담습니다. 그래서 트랜잭션을 되돌리는 rollback과, 다른 세션이 변경 전 시점의 데이터를 보게 하는 읽기 일관성 재구성에 쓰입니다.

①는 이 두 축을 인과로 뒤섞었습니다. roll forward(커밋된 변경의 재적용)는 전적으로 Redo의 몫이지 Undo의 역할이 아닙니다. Undo의 이전 이미지는 앞으로 나아가는 복구가 아니라 뒤로 되돌리는 rollback·읽기 일관성에 관여합니다. "Undo가 roll forward의 입력이 된다"는 서술은 Redo가 맡은 기능을 Undo에 잘못 옮겨 붙인 것입니다.

④·②·③은 각 구조의 역할을 축에 맞게 옮겨 진술합니다. 버퍼 캐시는 블록 캐싱(④), Redo는 roll forward 근거(②), Undo는 rollback·읽기 일관성(③)입니다.

선택지	판정	이유
①	×	roll forward는 Redo가 담당합니다. Undo의 이전 이미지는 rollback·읽기 일관성에 쓰일 뿐, 커밋분을 다시 적용하는 입력이 아닙니다. 두 구조의 역할 축을 뒤섞은 서술입니다
②	○	Redo 로그 버퍼의 변경 벡터는 로그 파일로 기록되어 장애 복구 시 커밋분을 재현하는 roll forward의 근거가 됩니다
③	○	Undo의 이전 이미지는 롤백과 읽기 일관성(변경 전 데이터 재구성) 양쪽에 쓰입니다. 옳은 서술입니다
④	○	DB 버퍼 캐시는 읽어 온 블록을 메모리에 담아 두어 재참조 시 물리 I/O를 줄이는 공간입니다. 역할이 옳습니다

▪ 한 줄 정리 커밋된 변경을 다시 적용하는 roll forward는 Redo의 역할이며, Undo의 이전 이미지는 rollback과 읽기 일관성에 쓰일 뿐 roll forward의 입력이 아닙니다.

문제 2. 정답 ④

I/O 방식과 대기 이벤트의 짝은 이렇게 고정되어 있습니다. 이벤트 이름이 직관과 반대라 이 짝을 외워 두어야 합니다.

Single Block I/O (한 번에 한 블록, 임의 위치)

→ 대기 이벤트: db file sequential read

→ 전형: 인덱스 Random 액세스 (ROWID로 테이블 블록 하나씩)

Multiblock I/O (한 Call에 인접 블록 여러 개)

→ 대기 이벤트: db file scattered read

→ 전형: Full Table Scan, Index Fast Full Scan

④은 이 짝에 정확히 부합합니다. 인덱스에서 얻은 ROWID로 테이블 블록을 한 건씩 임의로 찾아 읽는 것은 Single Block I/O이고, 그동안 관측되는 이벤트가 바로 (이름과 달리) **db file sequential read**입니다.

나머지는 이벤트 이름을 반대 방식에 붙였거나(②①), 오라클 이벤트와 SQL Server 이벤트를 서로 뒤바꿔(③) 놓았습니다.

PAGEIOLATCH 계열은 SQL Server의 I/O 대기 이벤트이고, **db file sequential/scattered read**는 오라클 고유의 이벤트입니다.

선택지	판정	이유
①	×	db file scattered read 는 인접 블록을 묶어 읽는 Multiblock I/O(Full Scan)에서 발생합니다. "한 블록씩 임의로 읽는다"는 Single Block I/O의 성질을 잘못 붙인 서술입니다
②	×	Multiblock I/O(Full Scan)에서 발생하는 이벤트는 db file scattered read 입니다. sequential read 를 붙인 것은 이벤트 이름을 반대 방식에 옳긴 서술입니다
③	×	PAGEIOLATCH_SH 는 SQL Server의 대기 이벤트이고 db file scattered read 는 오라클 것입니다. 두 DBMS의 이벤트를 서로 뒤바꿔 놓았습니다
④	○	ROWID로 테이블 블록을 한 건씩 임의로 읽는 것은 Single Block I/O이며, 그때 db file sequential read 가 관측됩니다. 짝이 정확합니다

- 한 줄 정리 ROWID로 테이블을 한 블록씩 찾아 읽는 Single Block I/O에서 관측되는 이벤트는 **db file sequential read**이며, **scattered read**(Multiblock)와 SQL Server의 **PAGEIOLATCH**를 뒤섞으면 안 됩니다.

문제 3. 정답 ④

배치가 계량기번호를 리터럴로 이어붙여 SQL을 만들면 대(臺)마다 SQL 문자열이 달라져 라이브러리 캐시에서 공유되지 못하고, 매 실행이 하드파싱을 유발합니다. 트레이스의 두 숫자가 이를 그대로 보여줍니다.

하드파싱 비율 = 라이브러리 캐시 미스 ÷ Parse 횟수
= 48,000 / 48,000 = 100% ← 파싱이 전부 하드파싱

파싱 CPU 비중(비수행문) = Parse CPU / Total CPU
= 42.510 / 53.860 × 100 ≈ 78.9% ← CPU의 약 4/5가 파싱

Parse Count(48,000)가 Execute Count(48,000)와 같다는 것은 실행마다 새로 파싱했다는 뜻이고, 미스가 파싱 횟수와 같으니 소프트파싱이 한 번도 걸리지 않았습니다. 게다가 하드파싱은 디셔너리 조회용 재귀 SQL을 대량으로 부릅니다.

재귀 파싱 배수 = 재귀 Parse / 비수행 Parse = 336,000 / 48,000 = 7배
전체 CPU 중 파싱 = (42.510 + 18.640) / (53.860 + 43.840)
= 61.150 / 97.700 × 100 ≈ 62.6%

즉 이 배치의 CPU는 데이터를 처리하는 데가 아니라 똑같은 모양의 SQL을 매번 새로 파싱하는 데 쏠리고 있습니다. 처방은 리터럴을 바인드 변수로 바꿔 하나의 공유 커서를 재사용(소프트파싱)하는 것입니다. 따라서 ④가 문제와 처방을 옳게 짚었습니다.

선택지	판정	이유
①	×	파싱 비용(하드/소프트)과 실행계획 품질(통계·히스토그램)은 서로 독립인 축입니다. 통계 수집은 오히려 커서를 무효화해 하드파싱을 늘릴 수 있고, 미스=Parse=48,000의 원인인 리터럴 문제를 건드리지 못합니다
②	×	Disk 820이 낮은 것은 맞지만, 대기가 아니라 CPU(53.86초 중 42.51초)가 파싱에 묶인 것이 병목입니다. Fetch Elapsed 12.47초를 네트워크 탭으로 돌리면 파싱 부하를 놓칩니다
③	×	재귀 Parse 336,000(=48,000×7)은 하드파싱이 디셔너리를 조회하며 파생시킨 증상입니다. 함수·트리거가 원인이 아니므로 그것을 제거해도 리터럴 하드파싱은 그대로입니다
④	○	미스 48,000 = Parse 48,000으로 100% 하드파싱이고 파싱이 비수행 CPU의 약 79%입니다. 바인드 변수로 커서를 공유해 소프트파싱으로 돌리는 것이 정확한 처방입니다

- 한 줄 정리 리터럴 SQL이 매 실행 하드파싱을 부르면 미스=Parse(100%)로 CPU의 약 79%가 파싱에 묶이고, 처방은 통계 수집이 아니라 바인드 변수로 커서를 공유하는 것입니다.

문제 4. 정답 ②

먼저 BCHR로 논리 대비 물리 읽기의 비율을 확인합니다.

BCHR = ((Query + Current) - Disk) / (Query + Current) × 100
= (1,250,000 + 0 - 150,000) / 1,250,000 × 100
= 1,100,000 / 1,250,000 × 100 = 88.0%
디스크 물리 읽기 비중 = 150,000 / 1,250,000 × 100 = 12.0%

논리 125만 블록 중 12%를 디스크에서 물리적으로 읽었습니다. 다음은 시간의 정체입니다.

CPU 비중 = 6.824 / 44.304 × 100 ≈ 15.4% → 나머지 약 84.6%가 대기
Elapsed - CPU = 44.304 - 6.824 ≈ 37.48초 ← 일한 시간이 아니라 기다린 시간

응답시간의 대부분이 대기이고, 그 대기가 어디서 났는지는 Row Source가 지목합니다.

결제 cr 비중 = 1,180,000 / 1,250,000 × 100 ≈ 94.4% ← 블록 소비의 대부분
결제 pr 비중 = 142,000 / 150,000 × 100 ≈ 94.7% ← 물리 읽기의 대부분

결제 time = 40.18초 (전체 Elapsed 44.3초의 대부분)

블록 소비(cr), 물리 읽기(pr), 소요 시간(time)이 세 지표 모두 결제 Full Scan 한 노드에 몰려 있습니다. 가맹점 Full Scan은 cr 70,000(5.6%)·pr 8,000에 불과합니다. 따라서 병목은 5,000만 건 결제 테이블의 Full Scan I/O이고, ②이 BCHR 해석과 병목 지목을 모두 옳게 했습니다.

선택지	판정	이유
①	×	CPU 6.82초는 Elapsed 44.3초의 약 15%뿐이라 '대부분'이 아닙니다. 응답시간의 약 85%가 대기이므로 CPU 바운드로 보고 DOP만 올리는 것은 원인 오귀속입니다
②	○	(1,250,000-150,000)/1,250,000 = 88.0%로 BCHR이 정확하고, cr·pr·time이 모두 결제 Full Scan(94% 이상)에 집중되어 병목 지목이 맞습니다
③	×	BCHR 88%는 맞지만, 5,000만 건을 훑는 Full Scan의 논리 읽기(cr=1,180,000)는 캐시를 키운다고 사라지지 않습니다. 캐시 부족이 아니라 스캔 물량 자체가 원인입니다
④	×	인덱스 스캔이 없는 것은 사실이나, 병목은 cr 5.6%뿐인 가맹점이 아니라 cr 94.4%의 결제 Full Scan입니다. 가맹점 인덱스는 44초 대기를 겨냥하지 못합니다

- 한 줄 정리 BCHR 88%와 CPU 15%까지는 옳게 읽더라도, cr·pr·time이 모두 결제 Full Scan에 몰린 사실을 놓치면 병목 지목이 틀립니다.

문제 5. 정답 ③

예상 계획 도구를 고를 때의 핵심 갈림길은 "문장을 실제로 돌리는가" 하나입니다. 도구마다 이 경계가 다릅니다.

문장 미수행 (예상 계획만)

오라클 : EXPLAIN PLAN FOR <문장> → PLAN_TABLE 적재 (추정치)
 SET AUTOTRACE TRACEONLY EXPLAIN
 SQL Server : SET SHOWPLAN_TEXT / SHOWPLAN_ALL / SHOWPLAN_XML ON

문장 수행함 (실제 계획, 실행 행 수 포함)

오라클 : SET AUTOTRACE TRACEONLY STATISTICS (STATISTICS를 켜면 수행)
 SQL Server : SET STATISTICS PROFILE / STATISTICS XML ON

③는 SQL Server 두 도구의 경계를 정확히 같았습니다. **SHOWPLAN_ALL ON**은 문장을 수행하지 않고 추정 계획만 내고, **STATISTICS PROFILE ON**은 문장을 실제로 수행해 실행 행 수가 담긴 실제 계획을 냅니다. "예상은 미수행, 실제는 수행"이라는 원칙 그대로입니다.

나머지는 이 경계를 뭉개줍니다.

①은 **TRACEONLY**에 **STATISTICS**까지 켜면 통계를 모으기 위해 **SELECT**문을 실제로 수행한다는 사실을 놓쳤습니다. 문장을 돌리지 않는 것은 **TRACEONLY EXPLAIN**(계획만)일 때뿐입니다. 결과 행을 화면에 안 찍는 것과 문장을 안 돌리는 것은 별개입니다.

②는 **SHOWPLAN**을 실행 도구로 오해했습니다. **SHOWPLAN_***은 미수행 예상 계획 도구입니다.

④은 **EXPLAIN PLAN**이 문장을 수행해 실행치를 담는다고 했지만, **EXPLAIN PLAN**은 문장을 돌리지 않고 추정 계획만 **PLAN_TABLE**에 적재합니다. 실행 행 수를 보려면 **gather_plan_statistics** 힌트로 실제 수행한 뒤 **DBMS_XPLAN.DISPLAY_CURSOR**를 써야 합니다.

선택지	판정	이유
①	×	TRACEONLY 에 STATISTICS 가 켜져 있으면 통계 수집을 위해 SELECT 문을 실제로 수행합니다. 결과 행 미출력과 문장 미수행은 다릅니다. 미수행은 TRACEONLY EXPLAIN 일 때뿐입니다
②	×	SHOWPLAN_TEXT ON 은 문장을 수행하지 않고 예상 계획만 반환합니다. 실제 수행 도구로 서술한 것은 경계를 뒤집은 것입니다
③	○	SHOWPLAN_ALL ON 은 미수행 예상 계획, STATISTICS PROFILE ON 은 수행 후 실제 계획을 냅니다. 두 도구의 경계가 정확합니다
④	×	EXPLAIN PLAN 은 문장을 수행하지 않고 추정 계획만 적재합니다. 실행 행 수는 담기지 않으며, 그것을 보려면 DISPLAY_CURSOR 가 필요합니다

- 한 줄 정리 **SHOWPLAN_***은 문장을 돌리지 않는 예상 계획, **STATISTICS PROFILE**은 돌리는 실제 계획이며, **AUTOTRACE TRACEONLY**도 **STATISTICS**가 켜지면 문장을 실제로 수행한다는 경계가 관건입니다.

문제 6. 정답 ②

플랜의 두 노드에서 나온 Card 값을 나란히 놓습니다.

INDEX RANGE SCAN 청구_IX Card 8,400 ← 인덱스에서 걸러진 건수
TABLE ACCESS BY INDEX ROWID 청구내역 Card 2,100 ← 테이블까지 거친 뒤 남은 건수

청구_IX는 (요금제코드, 청구월) 순서입니다. WHERE에는 요금제코드 = 'LTE52'(등치)와 청구월 범위가 있으므로, 선두 컬럼 등치 + 두 번째 컬럼 범위까지가 인덱스로 연속해서 훑을 수 있는 구간입니다. 그 결과가 INDEX RANGE SCAN의 8,400건입니다.

납부상태는 **청구_IX**의 구성 컬럼이 아닙니다. 따라서 인덱스만으로는 걸러낼 수 없고, ROWID로 테이블 블록을 읽어 온 TABLE ACCESS BY INDEX ROWID 단계에서 필터로 처리됩니다. 그래서 8,400건이 이 단계를 거치며 2,100건으로 줄어듭니다.

8,400 (인덱스 액세스: 요금제코드=, 청구월 범위)
└─ 테이블 필터(납부상태='미납') 적용 → 2,100 (최종)

즉 ②이 이 구조를 정확히 서술합니다. 납부상태까지 인덱스 액세스로 뚫고 그린 ①, 인덱스 반환 건수를 최종 건수로 본 ③, 선두 컬럼 없이도 RANGE SCAN이 성립한다는 ④는 모두 이 Card 흐름·인덱스 구조와 어긋납니다.

선택지	판정	이유
①	×	납부상태는 청구_IX 구성 컬럼이 아니므로 인덱스 액세스 조건이 될 수 없습니다. 세 조건 모두 인덱스로 처리했다면 INDEX RANGE SCAN Card가 이미 2,100이었을 것인데 실제로는 8,400입니다
②	○	요금제코드(등치)·청구월(범위)은 인덱스 액세스 조건, 납부상태는 인덱스에 없어 TABLE ACCESS 단계 필터 — Card가 8,400→2,100으로 줄어드는 흐름과 일치합니다
③	×	8,400은 인덱스 단계 Card일 뿐이고, 납부상태 필터를 거친 최종 Card는 2,100입니다. TABLE ACCESS BY INDEX ROWID 단계에서 건수 감소가 분명히 일어납니다
④	×	선두 컬럼 요금제코드에 조건이 없으면 인덱스의 정렬 시작점을 잡을 수 없어 청구월 범위만으로는 INDEX RANGE SCAN이 성립하지 않습니다. 선두 컬럼 등치가 있어야 8,400 구간이 만들어집니다

▪ 한 줄 정리 선두 컬럼 등치와 다음 컬럼 범위만 인덱스 액세스 조건이 되어 8,400건을 훑고, 인덱스에 없는 납부상태는 테이블 액세스 단계 필터가 되어 Card가 2,100으로 줄어듭니다.

문제 7. 정답 ②

트레이스의 세 숫자를 나란히 놓습니다.

620,000 ← 인덱스에서 얻은 ROWID 수 (INDEX RANGE SCAN 의 Rows)
211,340 ← 테이블 액세스 누적 블록 I/O (TABLE ACCESS 의 cr)
1,340 ← 인덱스 스캔에 든 블록 (INDEX RANGE SCAN 의 cr)

클러스터링 팩터(CF)는 인덱스 순서로 테이블을 읽을 때 발생하는 블록 I/O 수로, 이론상 최솟값은 테이블 블록 수(90,000), 최댓값은 테이블 행 수(9,000,000)입니다.

테이블 랜덤 액세스 블록 = 211,340 - 1,340 = 210,000
CF 밀도 ≈ 테이블 랜덤 블록 ÷ ROWID 수 = 210,000 / 620,000 ≈ 0.34 (ROWID당 블록)
CF 추정 ≈ 0.34 × 9,000,000 ≈ 3,060,000
→ 테이블 블록 90,000의 약 34배 (최솟값의 34배)
→ 나쁘지만 최댓값(900만)에 붙은 최악은 아닌 중간값

이 정도 CF에서 인덱스 경로의 실제 비용을 손익분기점 관점으로 따집니다.

읽은 행 비율 = 620,000 / 9,000,000 ≈ 6.9% ← 흔히 '인덱스가 유리'로 보이는 범위
인덱스 경로 블록 I/O = 210,000(테이블 랜덤, 대부분 단일블록) + 1,340(인덱스) = 211,340
Full Scan 예상 블록 = 90,000 (multiblock 읽기)
→ 211,340 > 90,000: 이 범위에선 손익분기점을 넘어 Full Scan이 유리

읽은 행이 7%에 불과해도, 나쁜 CF 탓에 단일블록 랜덤 읽기 21만이 Full Scan 9만 블록을 크게 웃돕니다. Elapsed 33.9초 대비 CPU 5.65초의 약 28초 간극은 물리 읽기(Disk 50,000) 대기이며, 그 뿌리는 테이블 랜덤 액세스입니다. 처방은 인덱스 재정렬이 아니라 Full Scan 전환 또는 필요한 컬럼을 인덱스에 얹은 커버링(테이블 액세스 제거)입니다. 따라서 ②가 옳은 진단입니다.

선택지	판정	이유
①	×	인덱스 cr=1,340이 작은 것은 맞지만 33초는 Array Size가 아니라 물리 읽기 대기입니다. Fetch 6,201회는 62만 행을 나눈 것일 뿐, 병목은 테이블 랜덤 블록 210,000입니다
②	○	테이블 랜덤 블록 210,000 / ROWID 620,000 $\approx$ 0.34로 CF는 테이블 블록의 약 34배(나뉘)이고, 랜덤 읽기 211,340 > Full Scan 90,000이라 손익분기점을 넘겨 Full Scan이 유리합니다
③	×	ROWID당 0.34는 최솟값 밀도(90,000/9,000,000 $\approx$ 0.01)의 34배로 오히려 나쁜 쪽입니다. 33초는 리프 블록 분할이 아니라 테이블 랜덤 액세스의 물리 I/O 대기입니다
④	×	(211,340-50,000)/211,340 $\approx$ 76%로 BCHR 계산은 맞지만, 낭비의 정체는 캐시가 아니라 논리 읽기 211,340 자체(나쁜 CF)입니다. 캐시를 키워도 21만 블록 액세스는 남습니다

- 한 줄 정리 ROWID 62만에 테이블 블록 21만이면 CF는 테이블 블록의 약 34배로 나뉘고, 7% 범위라도 랜덤 21만 > Full Scan 9만이라 손익분기점을 넘겨 Full Scan이 유리합니다.

문제 8. 정답 ④

인덱스가 Range Scan을 쓰려면 조건절의 컬럼이 가공되지 않은 원래 모습이어야 합니다. 인덱스는 컬럼의 원본 값 순서로 정렬되어 있어서, 컬럼을 건드리는 순간 그 정렬을 근거로 시작·끝점을 잡을 수 없기 때문입니다.

where col = :v → 원본 정렬 활용 → Range Scan 가능
 where f(col) = :v → 가공값의 순서 모름 → 시작·끝점 못 잡음 → Range Scan 불가
 where col + 1 = :v → 컬럼 연산도 가공 → 불가
 where char_col = 100 → 컬럼에 TO_NUMBER 묵시 적용 → 가공과 동일 → 불가

④은 이 원리를 정반대로 뒤집었습니다. 인덱스 컬럼에 함수를 씌우면 옵티마이저가 알아서 되돌려 Range Scan이 유지된다는 것은 사실이 아닙니다. 컬럼이 가공되면 일반적으로 그 인덱스를 정상적으로 타지 못하며, 이를 살리려면 개발자가 조건식을 바꾸거나(예: **col = :v/1** 형태로 상수 쪽만 가공) 함수 기반 인덱스(FBI)를 별도로 생성해 두어야 합니다. 옵티마이저가 임의로 가공을 풀어 주는 것이 아닙니다.

①·②·③는 모두 옳습니다. 선두 컬럼 가공은 스캔 범위를 못 잡고(①), 문자 컬럼에 숫자 상수를 비교하면 컬럼 쪽이 묵시적으로 형변환되어 인덱스가 무효화되며(②), **IS NULL**·부정형 조건은 시작점을 확정하지 못해 정상 Range Scan에 부적합합니다(③).

선택지	판정	이유
①	○	선두 컬럼을 함수·연산으로 가공하면 가공값의 정렬 순서를 알 수 없어 스캔 시작·끝점을 못 잡습니다. 옳은 서술입니다
②	○	char_col = 100 은 우선순위가 높은 숫자에 맞춰 컬럼에 TO_NUMBER 가 묵시 적용되어 컬럼 가공과 같아지므로 인덱스가 무효화됩니다
③	○	단일 컬럼 인덱스는 NULL을 저장하지 않아 IS NULL 로 행을 찾기 어렵고, 부정형은 시작점을 확정하지 못해 정상 Range Scan에 부적합합니다
④	×	방향이 뒤집혔습니다. 컬럼을 가공하면 옵티마이저가 되돌려 주지 않아 Range Scan을 잃습니다. 살리려면 조건식 수정이나 함수 기반 인덱스가 필요합니다

- 한 줄 정리 인덱스 컬럼을 함수·연산·묵시적 형변환으로 가공하면 Range Scan을 잃으며, 옵티마이저가 이를 자동으로 되돌려 유지해 주는 것이 아닙니다.

문제 9. 정답 ①

결합 인덱스는 선행 컬럼부터 정렬되고, 선행 컬럼 값이 같은 구간 안에서만 후행 컬럼이 정렬됩니다. 그래서 후행 컬럼이 스캔 범위를 좁히는 액세스 조건이 될 수 있는지는, 선행 컬럼이 한 점으로 고정되었는지에 달려 있습니다.

(A=, B>) : A가 한 점 → 그 안에서 B 정렬이 살아 있음 → B가 시작·끝점 좁힘 (액세스)
 (A>, B=) : A가 여러 값에 열림 → 각 A값마다 B 위치가 흩어짐 → B는 걸러내기만 (필터)

①는 이 경계를 오해했습니다. 선행 컬럼 A가 범위 조건이면 A는 이미 여러 값 구간에 걸쳐 열려 있고, 그 구간들 안에서 B는 연속으로 모여 있지 않습니다. 따라서 B에 등치 조건이 있어도 스캔 시작·끝점을 확정하지 못하고, A 범위에 해당하는 인덱스 구간을 전부 훑으면서 B 조건에 맞지 않는 엔트리를 필터로 버릴 뿐입니다. "B가 시작·끝점을 확정해 좁은 구간만 읽는다"는 서술은 선행 컬럼이 등치일 때만 성립하는 성질을 범위 조건 상황에 잘못 끌어다 붙인 것입니다.

④·②·③는 옳습니다. 선행 등치 + 후행 범위는 잘 좁혀지고(④), 선행 범위면 후행은 필터로만 작동하며(②), Skip Scan은 선행 컬럼 값 종류가 적을 때 성립하는 예외 경로입니다(③).

선택지	판정	이유
①	×	선행 A가 범위면 그 구간 안에서 B가 연속으로 모이지 않아, 후행 B가 등치여도 시작·끝점을 확정하지 못하고 필터로만 작동합니다. 등치 상황의 성질을 범위 상황에 잘못 옮긴 서술입니다
②	○	선행 A가 범위면 여러 값에 열려 후행 B는 시작·끝점을 못 정하고 필터로만 작동합니다. 원리에 맞는 서술입니다
③	○	Skip Scan은 선행 컬럼의 값 종류가 적고 후행에 조건이 있을 때 각 구간을 건너뛰며 탐색하는 예외 경로입니다. 옳습니다
④	○	A=로 한 점이 고정되면 그 안에서 B 범위가 정렬 순서대로 이어져 스캔 범위가 효율적으로 좁혀집니다. 옳은 서술입니다

- 한 줄 정리 선행 컬럼이 범위 조건이면 후행 컬럼은 등치라도 스캔 범위를 좁히지 못하고 필터로만 작동하며, "후행 등치가 구간을 최소화한다"는 것은 선행 등치일 때만 맞는 이야기입니다.

문제 10. 정답 ①

MERGE JOIN(Id 1) 아래 두 자식의 연산 종류를 읽습니다.

Id 1 MERGE JOIN ← 정렬된 두 스트림을 병합
 Id 2 SORT JOIN → TABLE ACCESS FULL 등급기준 ← 왼쪽 입력을 정렬
 Id 4 SORT JOIN → TABLE ACCESS FULL 평가점수 ← 오른쪽 입력을 정렬

두 입력이 각각 독립적으로 TABLE ACCESS FULL로 읽힌 뒤 SORT JOIN으로 정렬되고, 그 정렬 순서를 따라 MERGE JOIN이 두 스트림을 병합합니다. 이것이 소트 머지 조인의 표준 형태입니다.

조인 조건 **평가점수 BETWEEN 하한점수 AND 상한점수**는 비등치(범위) 조건입니다. 해시 조인은 해시 버킷을 등치(=)로만 매칭하므로 이런 범위 조인을 처리하지 못합니다. 반면 소트 머지 조인은 양쪽을 정렬해 두면 범위 비교로도 병합할 수 있어, 비등치 조인에서 유효한 선택지입니다. Id 4 SORT JOIN에 붙은 * 마커가 바로 이 범위 조인 조건이 평가되는 지점입니다. 그래서 ①이 이 플랜의 연산 구조(양쪽 독립 액세스 + 각각 정렬 + 병합)와 조인 조건 성격(비등치)을 모두 정확히 서술합니다.

②는 이 플랜에 없는 HASH JOIN을 끌어옵니다. 노드는 MERGE JOIN이고 해시 영역·탐침 노드가 없습니다.

③은 '정렬=비효율'이라는 반사적 판단인데, 조인 인덱스가 아예 없어 반복 탐색형 NL이 오히려 더 나쁠 수 있습니다.

④는 MERGE JOIN을 등치 전용으로 착각한 것으로, SORT MERGE는 비등치 조인을 처리하는 대표적 방식입니다.

선택지	판정	이유
①	○	Id 2·Id 4의 SORT JOIN이 양쪽을 정렬하고 Id 1 MERGE JOIN이 병합하며, 조인 조건이 BETWEEN(비등치)이라 해시 조인이 못 쓰이는 상황과 일치합니다
②	×	플랜에 HASH JOIN·해시 버킷 적재·탐침 노드가 없습니다. 상위 조인 노드는 MERGE JOIN이며 SORT JOIN은 해시가 아니라 병합을 위한 정렬입니다
③	×	두 테이블 어디에도 조인 인덱스가 없어 NL은 인너를 Full Scan으로 반복해야 합니다. 정렬이 있다는 사실만으로 NL보다 느리다고 단정할 수 없습니다
④	×	조인 조건은 평가점수 BETWEEN 하한 AND 상한 인 비등치이며, MERGE JOIN은 이런 범위 조인을 처리하는 방식입니다. 등치로 변환되지 않았고 Id 4의 *가 범위 조건 평가 지점입니다

- 한 줄 정리 MERGE JOIN 아래 두 SORT JOIN이 양쪽을 각각 정렬해 병합하는 소트 머지 조인이며, 조인 조건이 BETWEEN(비등치)이라 해시 조인이 못 쓰이는 상황에 적합합니다.

문제 11. 정답 ②

해시 조인은 Build → Probe 두 단계로 움직입니다.

Build 단계 : 한쪽 입력(Build Input) → Hash Area에 해시 테이블 구축
 Probe 단계 : 다른 쪽 입력(Probe Input) → 해시 테이블을 탐색해 매칭

이 구조에서 비용을 좌우하는 것은 해시 테이블이 메모리(Hash Area)에 통째로 들어가느냐입니다. 따라서 옵티마이저는 두 입력 중 더 작을 것으로 추정되는 쪽을 Build Input으로 잡습니다. 작은 쪽을 메모리에 올려야 해시 테이블이 Hash Area를 넘지 않고, Temp로 넘치는 one-pass·multi-pass를 피할 수 있기 때문입니다.

②는 이 기준을 표준 원리 목록 밖의 진술로 채웠습니다. "더 큰 쪽을 Build Input으로 삼는다"는 것은 방향이 반대이며, Probe 반복 횟수를 줄이려는 근거도 성립하지 않습니다. 해시 조인의 Probe는 NL 조인처럼 매 행 반복 탐색하는 것이 아니라 해시 테이블 조회 한 번으로 매칭되므로, "큰 쪽을 Build로 잡아 반복을 줄인다"는 인과 자체가 없는 이야기입니다. Build Input을 크게 잡으면 해시 테이블이 Hash Area를 넘겨 Temp 분할만 찾아질 뿐입니다.

①·③·④는 해시 조인의 표준 정리에 그대로 대응합니다. Build/Probe 2단계 구조(①), 등치 조인 성립(③), Hash Area 초과 시 Temp를 쓰는 one-pass·multi-pass 저하(④)가 모두 옳습니다.

선택지	판정	이유
①	○	Build 단계에서 해시 테이블을 Hash Area에 구축하고 Probe 단계에서 다른 입력을 훑어 매칭하는 2단계 구조를 정확히 설명합니다
②	×	작은 쪽을 Build Input으로 삼아야 해시 테이블이 Hash Area에 들어갑니다. 큰 쪽을 Build로 잡는다는 것은 표준 원리와 반대이며, Probe는 해시 조회 한 번이라 "반복 탐색을 줄인다"는 근거도 없습니다
③	○	해시 함수로 같은 값을 같은 버킷에 모으므로 등치(=) 조인 조건에서 성립하며, 부등호·범위 조인에는 쓰지 못합니다
④	○	Build Input이 Hash Area를 넘으면 일부 파티션을 Temp에 기록하는 one-pass·multi-pass로 처리되어 디스크 I/O로 성능이 저하됩니다

- 한 줄 정리 해시 조인은 작은 쪽을 Build Input으로 해시 테이블을 메모리에 짓고 큰 쪽으로 Probe하는 등치 조인 기법이므로, 큰 쪽을 Build Input으로 삼아 반복을 줄인다는 서술은 표준 원리 밖의 잘못된 진술입니다.

문제 12. 정답 ④

쿼리 변환은 저마다 목적과 방향이 다른 기법들입니다. 이름에서 연상되는 인상만으로 효과를 짚지으면 틀립니다.

조건절 Pushing(Predicate Pushing)은 이름 그대로 조건을 안쪽으로 밀어 넣는 변환입니다. 머지되지 못하는 뷰·인라인 뷰가 있을 때, 바깥 쿼리의 조건을 뷰 내부로 내려보내 뷰가 다뤄야 할 데이터를 일찍 걸러 냅니다. 처리량이 줄어드는 것이 이 변환의 이득입니다. ④이 이 원리를 정확히 진술합니다.

나머지는 이름에서 오는 반사적 연상을 노린 함정입니다.

①: Unnesting과 View Merging은 둘 다 "합친다"는 인상 때문에 같은 것으로 묶기 쉽지만, 대상이 다릅니다. Unnesting은 중첩 서브쿼리를 조인으로 평탄화하는 변환이고, View Merging은 뷰 정의를 바깥 쿼리에 풀어 넣는 변환입니다.

출발점(서브쿼리 vs 뷰)이 달라 같은 이름의 다른 표현이 아닙니다.

②: "Push(밀다)"에서 조인 순서를 밀어 고정한다고 연상하기 쉽지만, 조건절 Pushing은 조건을 미는 것이지 조인 순서를 고정하는 기법이 아닙니다. 오히려 조건이 안으로 들어가 데이터가 줄면 옵티마이저의 선택지는 넓어집니다.

③: 조건절 이행은 $A=B$ 이고 $B=C$ 이면 $A=C$ 를 유도해 새 조건을 만들어 내는 변환으로, 서브쿼리를 조인으로 푸는 Unnesting과는 전혀 다른 작업입니다. 이름을 서로 바꿔 붙인 오연결입니다.

선택지	판정	이유
①	×	Unnesting은 서브쿼리를 조인으로 평탄화, View Merging은 뷰를 바깥에 병합하는 변환입니다. "합친다"는 인상으로 둘을 같은 것으로 묶은 반사적 연상 오류입니다
②	×	Pushing은 조건을 안으로 미는 변환이지 조인 순서를 고정하는 기법이 아닙니다. 조건이 내려가면 오히려 탐색 폭이 넓어집니다
③	×	조건절 이행은 등치 관계로 새 조건을 유도하는 변환으로, 서브쿼리를 조인으로 바꾸는 Unnesting과 별개입니다. 이름을 뒤바꿔 붙인 오연결입니다
④	○	조건절 Pushing은 바깥 조건을 인라인 뷰 내부로 밀어 넣어 뷰가 처리할 행을 조기에 줄이는 변환입니다

- 한 줄 정리 조건절 Pushing은 바깥 조건을 뷰 안으로 밀어 넣어 처리량을 미리 줄이는 변환이며, Unnesting·View Merging·조건절 이행을 이름의 인상으로 뒤섞는 것은 반사적 연상 오류입니다.

문제 13. 정답 ①

라이브러리 캐시 요약이 그대로 답을 보여 줍니다. 논리적으로 같은 세 SQL인데 SQL_ID가 세 개(g4t7..., c8m3..., s2v9...)로 갈렸고, 각 행이 **PARSE CALLS 1 / EXECUTIONS 1 / LOADS 1**입니다. LOADS 1은 커서를 라이브러리 캐시에 새로 적재(하드파싱) 했다는 뜻입니다.

g4t7... ← 세션 P : 원본

c8m3... ← 세션 Q : 키워드만 대문자

s2v9... ← 세션 R : index 힌트 추가

→ 텍스트가 다르면 SQL_ID가 다르고, 기존 커서를 못 찾아 각각 새로 적재(LOADS 1)

오라클의 기본(EXACT) 커서 공유는 SQL 텍스트를 문자 단위로 비교해 라이브러리 캐시에서 부모 커서를 찾습니다. ①는 "옵티마이저가 비교 전에 텍스트를 대문자로 정규화하고 힌트·주석·여백을 제거한다"는 정규화 가정의 오류입니다. 기본 모드에서 오라클은 그런 정규화를 하지 않으므로, 대소문자·힌트·공백이 하나만 달라도 다른 텍스트로 간주되어 별도 커서를 새로 만들며 하드파싱합니다. 만약 ①처럼 정규화가 일어난다면 요약표의 SQL_ID는 하나여야 하는데, 실제로는 셋입니다.

그래서 부적절한 진술은 ①입니다.

④②③은 모두 "텍스트가 다르면 커서를 공유하지 못한다"는 실제 동작을 옳게 서술합니다. 특히 ③은 공유의 조건이 바인드 변수화만이 아니라 텍스트 전체의 동일성임을 정확히 짚습니다.

선택지	판정	이유
①	×	기본 커서 공유는 텍스트를 대문자화·힌트/주석/여백 제거로 정규화하지 않습니다. 정규화된다면 SQL_ID가 하나여야 하는데 요약표에는 셋입니다
②	○	힌트는 SQL 텍스트의 일부라 SQL_ID를 바꿉니다. 세션 R의 별도 SQL_ID(s2v9...)가 그 증거이며, 원본과 다른 커서를 씁니다
③	○	리터럴을 바인드 변수로 바꿔도 대소문자·힌트·공백이 다르면 텍스트가 여전히 달라 공유되지 않습니다. 텍스트 전체를 일치시켜야 소프트파싱됩니다
④	○	세 SQL은 의미가 같아도 텍스트가 문자 단위로 다릅니다. SQL_ID가 셋(g4t7·c8m3·s2v9)으로 갈리고 LOADS 1이 붙어 각각 하드파싱되었음을 요약표가 보여 줍니다

- 한 줄 정리 오라클 기본 커서 공유는 텍스트를 정규화하지 않고 문자 단위로 비교하므로, 대소문자·힌트·공백만 달라도 각각 하드파싱되어 SQL_ID가 갈립니다 — "정규화해 공유해 준다"는 것은 착각입니다.

문제 14. 정답 ③

힌트는 옵티마이저에 대한 지시(directive)이지만, 그 지시가 늘 관철되는 것은 아닙니다. 힌트의 처리 규칙을 정리하면 이렇습니다.

힌트 상태 → 옵티마이저의 처리

오타·문법 오류(유효하지 않음) → 조용히 무시(오류 아님)
 문법상 유효하나 실행 불가능한 경로 → 조용히 무시
 문법상 유효하고 실행 가능한 경로 → 지시대로 적용

핵심은 유효하지 않거나 적용 불가능한 힌트도 오류를 내지 않고 무시된다는 점입니다. 힌트는 주석(/*+ ... */) 형태로 전달되므로, 잘못 적혀 있어도 SQL 자체는 정상 실행되고 해당 힌트만 버려집니다. ③이 이 규칙을 정확히 담았습니다.

나머지는 부분적 진실을 전제로 확대하거나 힌트의 역할을 뒤바꾼 진술입니다.

②: "USE_NL을 주면 NL로 수행된다"는 것은 실행 가능할 때만 성립하는 부분 진실입니다. 예컨대 조인 키에 인덱스가 없어 NL이 사실상 불가능하거나 다른 힌트와 충돌하면 이 힌트는 무시됩니다. "무조건 수행된다"로 굳히면 무시 가능성을 지운 과장이 됩니다.

①: LEADING·ORDERED는 조인 순서를 지정하는 힌트이지, 액세스 경로(인덱스/풀스캔)를 정하는 힌트가 아닙니다. 액세스 경로는 INDEX·FULL 등이 담당합니다. 역할을 뒤바꾼 진술입니다.

④: 오타 난 힌트는 구문 오류가 아니라 무시됩니다. SQL은 그대로 실행되며 결과도 정상적으로 반환됩니다(계획만 힌트 없이 수립). "문법을 정확히 지켜야 실행된다"는 것은 ③의 정반대입니다.

선택지	판정	이유
①	×	LEADING·ORDERED는 조인 순서를 지정하는 힌트입니다. 액세스 경로 지정은 INDEX·FULL의 역할로, 힌트의 담당을 뒤바꿨습니다
②	×	USE_NL은 실행 가능할 때만 적용됩니다. 불가능하거나 충돌하면 무시되므로, "무조건 NL로 수행된다"는 무시 가능성을 지운 과장입니다
③	○	유효하지 않거나 실행 불가능한 힌트는 오류 없이 무시되고 SQL은 정상 실행됩니다. 힌트의 처리 규칙을 정확히 진술합니다
④	×	오타 난 힌트는 구문 오류가 아니라 무시되고 SQL은 정상 실행됩니다. "문법을 지켜야 실행된다"는 힌트 무시 규칙과 반대입니다

- 한 줄 정리 유효하지 않거나 실행 불가능한 힌트는 오류 없이 무시되고 SQL은 정상 실행되므로, 힌트가 지시대로 관철된다는 진술은 실행 가능 조건을 지운 부분 진실입니다.

문제 15. 정답 ①

소트 튜닝의 핵심 목표 하나는 불필요한 소트 오퍼레이션을 없애는 것입니다. 그리고 그 대표적 수단이 이미 정렬된 인덱스의 순서를 이용하는 것입니다.

인덱스는 키 값이 정렬된 상태로 저장되어 있습니다. 따라서 **ORDER BY** 대상 컬럼이 인덱스 정렬 순서와 일치하면, 옵티마이저는 인덱스를 순서대로 훑는 것만으로 정렬된 결과를 얻습니다. 이때 SORT ORDER BY 오퍼레이션은 추가되는 것이 아니라 생략(NOSORT)됩니다. 이것이 인덱스로 소트 부하를 없애는 원리입니다.

①는 이 원리를 정반대로 뒤집었습니다. "인덱스 순서로 처리하면서도 정합성을 위해 SORT ORDER BY를 한 번 더 추가한다"는 것은, 인덱스로 소트를 없애는 이득 자체를 부정하는 서술입니다. 이미 정렬된 것을 다시 정렬할 이유가 없으므로 소트는 붙지 않습니다.

②·③·④는 옳습니다. Sort Area 초과 시 Temp 디스크 소트로 저하되고(②), GROUP BY가 인덱스 정렬 순서와 맞으면 SORT GROUP BY를 생략하며(③), SORT UNIQUE·SORT AGGREGATE는 각각 중복 제거·전체 집계를 담당하는 소트 계열 오퍼레이션입니다(④).

선택지	판정	이유
①	×	방향이 뒤집혔습니다. 인덱스 정렬 순서가 ORDER BY와 맞으면 SORT ORDER BY는 추가되는 것이 아니라 생략(NOSORT)됩니다. 이미 정렬된 것을 다시 정렬하지 않습니다
②	○	정렬은 Sort Area(PGA)에서 이뤄지고, 초과분은 Temp를 쓰는 디스크 소트(multi-pass)로 처리되어 디스크 I/O로 성능이 저하됩니다
③	○	GROUP BY 컬럼이 인덱스 선두 컬럼의 정렬 순서와 맞으면 SORT GROUP BY 없이 인덱스를 훑어 그룹핑할 수 있습니다
④	○	SORT UNIQUE는 중복 제거, SORT AGGREGATE는 GROUP BY 없는 전체 집계를 위한 소트 계열 오퍼레이션입니다

▪ 한 줄 정리 인덱스의 정렬된 순서를 이용하면 ORDER BY·GROUP BY의 소트 오퍼레이션이 생략되므로, 인덱스로 처리하면서 소트가 추가된다는 서술은 방향을 뒤집은 정반대 진술입니다.

문제 16. 정답 ③

두 최적화는 줄이는 대상이 서로 다릅니다. 하나는 클라이언트-서버 왕복(Fetch Call = User Call), 다른 하나는 서브쿼리 블록의 반복 실행입니다.

[ArraySize 산출]

```

ArraySize = 반환 행 수 ÷ (Fetch Call - 1)    ← 마지막 no-more-data fetch의 -1
           = 1,500,000 ÷ (3,001 - 1)
           = 1,500,000 ÷ 3,000
           = 500

```

[두 최적화의 효과 구분]

```

Array Processing → Fetch Call(User Call) 횟수를 줄임 = 네트워크 왕복 감소
ArraySize 500 → 예: 1,000으로 키우면 Fetch ≈ 1,501회로 감소
스칼라 서브쿼리 캐싱 → 같은 dev_id 재조회를 캐시로 대체
device 조회를 최대 150만 → 40종(distinct) 수준으로 감소

```

ArraySize = 500입니다. 한 번의 Fetch Call로 500행씩 실어 나르므로 150만 행에 3,000번의 Fetch(+마지막 빈 Fetch 1) = 3,001회입니다. ArraySize를 더 키우면 Fetch Call이 줄어 User Call(왕복)이 감소합니다 — 이것이 Array Processing의 효과입니다.

스칼라 서브쿼리 캐싱은 왕복이 아니라, (**SELECT dev_nm ...**) 블록의 반복 실행을 줄입니다. dev_id가 40종뿐이므로 캐시가 히트해 device 조회가 최대 40회 수준으로 압축됩니다.

③은 ArraySize를 500으로 정확히 산출하고, 두 최적화의 효과(왕복 감소 vs 반복 조회 감소)를 올바르게 대응시켰습니다.

선택지	판정	이유
①	×	ArraySize는 50이 아니라 $1,500,000 \div 3,000 = 500$ 입니다. 또 ArraySize를 키우는 것은 Fetch 왕복을 줄일 뿐, device 조회를 줄이는 것은 스칼라 캐싱의 몫입니다
②	×	두 효과를 통째로 뒤바꿨습니다. Fetch Call을 줄이는 것은 Array Processing이고, device 조회를 40회로 줄이는 것은 스칼라 서브쿼리 캐싱입니다
③	○	$ArraySize = 1,500,000 \div (3,001 - 1) = 500$ 이 맞고, ArraySize ↑는 Fetch(User Call) 왕복을, 스칼라 캐싱은 device 조회를 40종 수준으로 줄인다는 효과 구분이 정확합니다
④	×	Fetch Call은 반환 행 수와 같지 않습니다. ArraySize 500 덕에 150만 행이 3,001회로 묶였습니다. ArraySize를 키우면 왕복은 더 줄어듭니다

- 한 줄 정리 ArraySize는 $1,500,000 \div (3,001-1) = 500$ 이며, ArraySize를 키우면 Fetch 왕복이 줄고(Array Processing), 40종뿐인 dev_id는 스칼라 서브쿼리 캐싱으로 device 조회가 40회 수준으로 줄어듭니다 — 두 효과는 대상이 다릅니다.

문제 17. 정답 ④

Partition Pruning은 옵티마이저가 WHERE 조건의 값을 파티션 경계(VALUES LESS THAN)와 대조해 스캔 대상을 미리 좁히는 최적화입니다. 이 대조가 가능하려면 조건이 파티션 키 칼럼(측정일시) 자체에 가공 없이 걸려 있어야 합니다.

- ① 측정일시 $\in [2025-07-01, 2025-10-01]$ → sp_2025q3 한 개
- ② 측정일시 $\in [2025-01-01, 2025-06-30]$ → sp_2025q1, sp_2025q2 두 개
- ③ 측정일시 $\geq 2026-01-01$ → sp_2026q1, sp_max (2025년 건너뛴)
- ④ TRUNC(측정일시) = 2025-08-15 → 키에 함수, 경계 대조 불가 → 전 파티션 스캔

①②③은 측정일시를 가공하지 않은 날짜 범위로 걸었으므로, 옵티마이저가 경계와 대조해 해당 파티션만 접근합니다.

②·③처럼 여러 파티션에 걸쳐도 나머지는 건너뛰므로 Pruning은 유효합니다.

④는 TRUNC(측정일시)로 파티션 키에 함수를 씌웠습니다. 그러면 옵티마이저는 TRUNC 결과값을 파티션 경계와 대조할 수 없어 Pruning을 포기하고, sp_2025q1 ... sp_max 전 파티션을 스캔한 뒤 각 행에서 TRUNC를 평가해 필터합니다. "8월 15일 하루만 겨냥한다"는 의미상의 부분적 진실이 있어도, 키 가공 한 줄이 최선(1개 파티션)을 최악(전 파티션 스캔)으로 뒤집습니다. 그래서 Pruning이 작동한다고 볼 수 없는 것은 ④이며, 이 진술이 적절하지 않습니다.

선택지	판정	이유
①	○	측정일시를 가공 없이 [2025-07-01, 2025-10-01] 범위로 걸어 sp_2025q3 한 개로 정확히 Pruning됩니다
②	○	BETWEEN 구간이 1~6월(2개 분기)에 걸쳐 sp_2025q1·sp_2025q2 두 파티션만 접근하고 나머지는 건너뛴다
③	○	$\geq 2026-01-01$ 상한 조건이 경계와 대조돼 sp_2026q1·sp_max만 읽고 2025년 네 파티션은 건너뛴다. Pruning 작동합니다
④	×	파티션 키에 TRUNC를 씌워 경계 대조가 불가능해집니다. 전 파티션을 스캔한 뒤 필터하므로 한 파티션만 읽는다는 진술은 성립하지 않습니다

- 한 줄 정리 측정일시를 가공하지 않은 날짜 범위는 Pruning이 작동하지만, TRUNC(측정일시)처럼 키에 함수를 씌우면 경계 대조가 불가능해져 전 파티션을 스캔합니다.

문제 18. 정답 ②

분배 방식은 각 생산자 TQ의 PX SEND 종류와, 조인 자식이 PX SEND를 거치는지로 읽습니다.

- Id 5 PX SEND BROADCAST :TQ10000 캠페인 쪽 생산자 → 전체 복제
- Id 7 TABLE ACCESS FULL 캠페인 (Q1,00) → 소형 빌드 입력
- Id 8 PX BLOCK ITERATOR (Q1,01) 노출이력을 조인 슬레이브가 직접 스캔
- Id 9 TABLE ACCESS FULL 노출이력 (Q1,01, Pstart 1~8) → PX SEND 없음

Id 5는 캠페인(Id 7)을 읽어 PX SEND BROADCAST로 내보냅니다 → 소형 테이블을 병렬 서버 전체에 복제. Id 4 PX RECEIVE가 이를 받아 HASH JOIN의 빌드 입력이 됩니다.

노출이력(Id 9)은 HASH JOIN과 같은 TQ(Q1,01) 아래에서 Id 8 PX BLOCK ITERATOR로 직접 읽힙니다. 그 위에 PX SEND/RECEIVE가 없습니다. 즉 대형 테이블은 재분배되지 않고, 각 병렬 서버가 자기 파티션 조각(Pstart 1~8)만 스캔해 곧바로 조인의 탐침(probe) 입력으로 씁니다.

소형만 복제하고 대형은 재분배하지 않는 이 방식이 broadcast-none 분배이며 ②의 서술 그대로입니다. 대형을 옮기지 않으므로 3억 건 재분배 비용이 사라지는 것이 이 방식의 이점입니다.

나머지 셋은 실제 노드와 값을 바꿔치기한 것입니다.

- 실제: 캠페인 = BROADCAST, 노출이력 = 재분배없음(로컬 스캔) (② broadcast-none)
- ① hash-hash : 캠페인 = HASH, 노출이력 = HASH → SEND BROADCAST·SEND 부재와 불일치
- ③ hash-broadcast: 노출이력 = HASH, 캠페인 = BROADCAST → 노출이력에 SEND HASH 없음
- ④ none-broadcast: 노출이력 = BROADCAST, 캠페인 = 로컬 스캔 → BROADCAST가 캠페인 쪽(Id 5)이라 대소를 뒤바꿈

선택지	판정	이유
①	×	PX SEND HASH 가 어디에도 없습니다. 캠페인 쪽은 SEND BROADCAST(Id 5)이고 노출이력 쪽은 PX SEND 자체가 없어 hash-hash가 아닙니다
②	○	Id 5가 PX SEND BROADCAST (캠페인)이고, 노출이력은 조인과 같은 Q1,01 아래 Id 8 PX BLOCK ITERATOR로 SEND 없이 로컬 스캔됩니다. 소형만 복제하는 broadcast-none 분배입니다
③	×	노출이력 쪽에 PX SEND HASH 가 없고 재분배 노드도 없습니다. 노출이력은 Q1,01에서 로컬 스캔되므로 해서 재분배가 아닙니다
④	×	BROADCAST 노드는 캠페인 쪽 Id 5입니다. 노출이력은 복제되지 않고 로컬 스캔되므로, 대형·소형의 역할을 뒤바꾼 진술입니다

- 한 줄 정리 캠페인 쪽만 **PX SEND BROADCAST**로 복제되고 노출이력은 조인과 같은 Q1,01에서 PX SEND 없이 자기 파티션만 로컬 스캔하므로, 소형만 복제하는 broadcast-none 분배입니다.

문제 19. 정답 ③

read-modify-write에서 두 세션이 갱신 전 같은 값을 읽고 각자 되쓰면, 나중 쓰기가 앞선 쓰기를 덮어서 갱신 하나가 사라집니다(Lost Update). 격리수준을 바꿔도 이 패턴은 막히지 않습니다.

[타임라인 - 잔여수량 초기 200]

```
t1 TX1 SELECT → 200
t2 TX2 SELECT → 200      (아직 아무도 커밋 안 함)
t3 TX1 UPDATE = 200 - 1 = 199 ; COMMIT
t4 TX2 UPDATE = 200 - 1 = 199 ; COMMIT ← TX1의 -1이 사라짐
```

기대값(정상 직렬화) = 200 - 1 - 1 = 198

실제 최종값(RC) = 199 ← Lost Update

실제 최종값(RCSI) = 199 ← 스냅샷 읽기도 read-modify-write의 갱신손실은 못 막음

기본 READ COMMITTED는 SELECT가 잡은 공유(S) 락을 읽은 직후 바로 해제합니다(트랜잭션 끝까지 유지하지 않음).

그래서 t1·t2에서 두 세션 모두 200을 읽을 수 있고, t3·t4의 UPDATE는 앞서 읽은 200을 기준으로 199를 씁니다. 결과는 199 - Lost Update입니다.

RCSI를 켜도 두 세션은 각자 스냅샷에서 200을 읽으므로 여전히 199가 됩니다. 즉 "커밋된 값 200을 읽었다"는 사실은 RC·RCSI 어디서나 참이라 격리수준을 가르는 근거가 못 됩니다. 이 값을 199→198로 뒤집으려면 격리수준이 아니라 잠금 읽기(UPDLCK)나 원자적 갱신(**SET 잔여수량 = 잔여수량 - 1**)이 필요합니다.

③가 현상(Lost Update)과 최종값(199)을 모두 맞혔습니다.

선택지	판정	이유
①	×	두 SELECT는 같은 행을 같은 값(200)으로 읽었습니다. 없던 행이 나타나는 Phantom Read가 아니며, 최종값도 198이 아니라 199입니다
②	×	READ COMMITTED의 S 락은 읽은 직후 해제되지 최종까지 유지되지 않습니다. 그래서 Lost Update가 차단되지 않고 최종값은 198이 아니라 199입니다
③	○	두 세션이 200을 읽고 각자 199로 되써 TX1의 차감이 사라졌습니다. 전형적 Lost Update이고 최종 잔여수량은 199입니다
④	×	'커밋된 값 200을 읽었다'는 RC·RCSI 공통 사실이라 판별 근거가 못 됩니다. RCSI를 켜도 두 세션이 200을 읽어 최종값은 여전히 199입니다

- 한 줄 정리 두 세션이 200을 읽고 각자 199로 되쓴 전형적 Lost Update로 최종 잔여수량은 199이며, READ COMMITTED든 RCSI든 이 read-modify-write 패턴의 갱신손실은 막지 못합니다.

문제 20. 정답 ③

블로킹은 동일 자원에서 한쪽은 락을 GRANT로 보유(LMODE≠0, BLOCK=1)하고 다른 쪽은 REQUEST로 대기(LMODE=0, REQUEST≠0) 할 때 성립합니다. 표를 자원별로 갈라 방향을 확인합니다.

자원	142	167	판정
TM 88231 (RX / RX)	LMODE 3	LMODE 3	RX↔RX 호환 → 대기 없음
TX 655370,942 (X / X요청)	LMODE 6, BLOCK1	REQUEST 6	X↔X 비호환 → 167 대기

TM(테이블) 락은 두 세션 모두 RX(모드 3)로 보유하며 RX끼리는 호환되므로 테이블 수준 대기는 없습니다(①이 옳음).

실제 충돌은 TX 자원(655370,942)에서 일어납니다. SID 142가 X(모드 6)를 GRANT로 잡고 BLOCK=1(다른 세션을 막는 중)이며, SID 167은 같은 TX 자원에 X를 REQUEST하고 LMODE=0으로 대기합니다. TX 락은 배타적(X)이라 X↔X는 비호환이므로 167이 142에 막힙니다.

③은 이를 정반대로 뒤집었습니다. TX 락을 "공유(S) 모드라 호환"이라고 했지만 TX는 배타(X) 모드로 비호환이고, 블로킹 방향도 "142가 167을 기다린다"고 했지만 실제로는 167이 142를 기다립니다(BLOCK=1이 142에 있음). 모드와 방향을 모두 반대로 진술했으므로 부적절한 것은 ③입니다.

선택지	판정	이유
①	○	TM 락은 두 세션 모두 RX(모드 3)이고 RX끼리 호환되므로 테이블 수준에서는 대기가 발생하지 않습니다
②	○	SID 167은 TX(655370,942)에 REQUEST 6·LMODE 0으로 대기 중이고, 같은 자원을 BLOCK=1로 잡은 SID 142에 막혀 있습니다
③	×	TX 락은 공유(S)가 아니라 배타(X, 모드 6)라 X↔X 비호환입니다. 또 BLOCK=1은 142에 있으므로 방향도 반대 — 142가 아니라 167이 기다립니다
④	○	같은 행을 두 세션이 갱신하려는 행 수준 TX 경합이며, 대기(REQUEST)는 167 한쪽에만 있으므로 단방향 블로킹이지 교착이 아닙니다

▪ 한 줄 정리 TM의 RX는 호환이라 대기가 없고, 배타적 TX(X)에서 BLOCK=1인 SID 142가 SID 167을 막고 있습니다 — TX를 공유 모드로 보거나 방향을 뒤집는 진술이 틀립니다.